

SEPT - TADS

Estruturas de Dados 2 - Quick-sort

Prof. Alex Kutzke

16 de outubro de 2018

Slides adaptados do material produzido
pelo Prof. Rodrigo Minetto
(<http://www.dainf.ct.utfpr.edu.br/~rminetto/>)

Quick Sort

- Quicksort é o algoritmo de ordenação interna mais rápido que se conhece para uma ampla variedade de situações;
- É provavelmente o algoritmo mais utilizado para ordenação;
- Inventado em 1960 (C.A.R. Hoare).
- Algoritmo de ordenação não estável.

Ordenação

Problema: ordenar um vetor em ordem crescente.

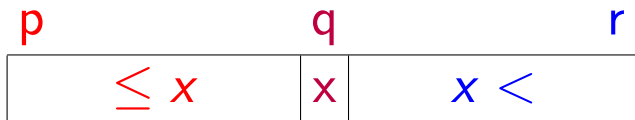
- **Entrada:** um vetor $A[0 \dots n-1]$
- **Saída:** vetor $A[0 \dots n-1]$
rearranjado em ordem crescente

Quick Sort - Ideia

Segue o paradigma de **divisão-e-conquista**:

Divisão: divida o vetor A em dois subvetores

$A[p \dots q - 1]$ e $A[q + 1 \dots r]$ tais que:



$$A[p \dots q - 1] \leq A[q] < A[q + 1 \dots r] \quad (1)$$

Conquista: ordene os dois subvetores

recursivamente utilizando o **Quick Sort**.

Partição

Problema: Rearranjar um dado vetor $A[p \dots r]$

e devolve um índice q , onde $p \leq q \leq r$, tais que:

$$A[p \dots q - 1] \leq A[q] < A[q + 1 \dots r] \quad (2)$$

Exemplo: entrada

p										r
99	33	55	77	11	22	88	66	33	44	

Saída:

p						q					r
33	11	22	33	44	55	88	66	77	99		

q : conhecido como pivô!

Exemplo de execução do particione

p					r				
99	33	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

									x	
p	99	33	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $\triangleright x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

<i>i</i>	<i>p</i>								<i>x</i>
99	33	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $\triangleright i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

<i>i</i>	<i>j</i>								<i>x</i>
99	33	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: $\triangleright$ Para $j \leftarrow p$ até $r - 1$ faça
- 5: Se $A[j] \leq x$ então
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: fim
- 9: fim
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: Retorne $i + 1$;

Exemplo de execução do particione

<i>i</i>	<i>j</i>								<i>x</i>
99	33	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: $\triangleright$ **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

<i>i</i>	<i>j</i>								<i>x</i>
99	33	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: ▷ Para $j \leftarrow p$ até $r - 1$ faça
- 5: Se $A[j] \leq x$ então
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: fim
- 9: fim
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: Retorne $i + 1$;

Exemplo de execução do particione

<i>i</i>	<i>j</i>								<i>x</i>
99	33	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: ▷ **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

<i>i</i>	<i>j</i>								<i>x</i>
99	33	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: ▷ $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

<i>i</i>	<i>j</i>								<i>x</i>
33	99	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $\triangleright A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

<i>i</i>		<i>j</i>							<i>x</i>
33	99	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: ▷ Para $j \leftarrow p$ até $r - 1$ faça
- 5: Se $A[j] \leq x$ então
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: fim
- 9: fim
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: Retorne $i + 1$;

Exemplo de execução do particione

<i>i</i>		<i>j</i>							<i>x</i>
33	99	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: ▷ **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

<i>i</i>			<i>j</i>						<i>x</i>
33	99	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: ▷ Para $j \leftarrow p$ até $r - 1$ faça
- 5: Se $A[j] \leq x$ então
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: fim
- 9: fim
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: Retorne $i + 1$;

Exemplo de execução do particione

<i>i</i>			<i>j</i>						<i>x</i>
33	99	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: ▷ **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

<i>i</i>				<i>j</i>					<i>x</i>
33	99	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: $\triangleright$ Para $j \leftarrow p$ até $r - 1$ faça
- 5: Se $A[j] \leq x$ então
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: fim
- 9: fim
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: Retorne $i + 1$;

Exemplo de execução do particione

<i>i</i>				<i>j</i>					<i>x</i>
33	99	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: $\triangleright$ **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

	i			j					x
33	99	55	77	11	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: $\triangleright i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

	<i>i</i>			<i>j</i>					<i>x</i>
33	11	55	77	99	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $\triangleright A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

	<i>i</i>				<i>j</i>				<i>x</i>
33	11	55	77	99	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: $\triangleright$ Para $j \leftarrow p$ até $r - 1$ faça
- 5: Se $A[j] \leq x$ então
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: fim
- 9: fim
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: Retorne $i + 1$;

Exemplo de execução do particione

	<i>i</i>				<i>j</i>				<i>x</i>
33	11	55	77	99	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: $\triangleright$ **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

		<i>i</i>			<i>j</i>				<i>x</i>
33	11	55	77	99	22	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: ▷ $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

		<i>i</i>			<i>j</i>				<i>x</i>
33	11	22	77	99	55	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $\triangleright A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

		<i>i</i>				<i>j</i>			<i>x</i>
33	11	22	77	99	55	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: ▷ Para $j \leftarrow p$ até $r - 1$ faça
- 5: Se $A[j] \leq x$ então
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: fim
- 9: fim
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: Retorne $i + 1$;

Exemplo de execução do particione

		<i>i</i>				<i>j</i>			<i>x</i>
33	11	22	77	99	55	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: $\triangleright$ **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

		<i>i</i>					<i>j</i>		<i>x</i>
33	11	22	77	99	55	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: ▷ Para $j \leftarrow p$ até $r - 1$ faça
- 5: Se $A[j] \leq x$ então
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: fim
- 9: fim
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: Retorne $i + 1$;

Exemplo de execução do particione

		<i>i</i>					<i>j</i>		<i>x</i>
33	11	22	77	99	55	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: ▷ **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

		<i>i</i>						<i>j</i>	<i>x</i>
33	11	22	77	99	55	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: $\triangleright$ Para $j \leftarrow p$ até $r - 1$ faça
- 5: Se $A[j] \leq x$ então
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: fim
- 9: fim
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: Retorne $i + 1$;

Exemplo de execução do particione

		<i>i</i>						<i>j</i>	<i>x</i>
33	11	22	77	99	55	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: $\triangleright$ **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

			i					j	x
33	11	22	77	99	55	88	66	33	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: ▷ $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

i				j						x
33	11	22	33	99	55	88	66	77	44	

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: ▷ $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

i				j						x
33	11	22	33	99	55	88	66	77	44	

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; ▷ x é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: ▷ Para $j \leftarrow p$ até $r - 1$ faça
- 5: Se $A[j] \leq x$ então
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: fim
- 9: fim
- 10: $A[i + 1] \leftrightarrow A[r]$;
- 11: Retorne $i + 1$;

Exemplo de execução do particione

p		i				r			
33	11	22	33	99	55	88	66	77	44

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $\triangleright A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Exemplo de execução do particione

p		q				r			
33	11	22	33	44	55	88	66	77	99

- 1: PARTICIONE(A, p, r)
- 2: $x \leftarrow A[r]$; $\triangleright x$ é o “pivô”
- 3: $i \leftarrow p - 1$;
- 4: **Para** $j \leftarrow p$ até $r - 1$ **faça**
- 5: **Se** $A[j] \leq x$ **então**
- 6: $i \leftarrow i + 1$;
- 7: $A[i] \leftrightarrow A[j]$;
- 8: **fim**
- 9: **fim**
- 10: $\triangleright A[i + 1] \leftrightarrow A[r]$;
- 11: **Retorne** $i + 1$;

Pseudo-código

```
1: Quick-Sort( $A, p, r$ )
2:   Se  $p < r$  então
3:      $q \leftarrow$  Particione( $A, p, r$ );
4:     Quick-Sort( $A, p, q - 1$ );
5:     Quick-Sort( $A, q + 1, r$ );
6:   fim
7:
```

Pseudo-código

```
1: Quick-Sort( $A, p, r$ )
2:   Se  $p < r$  então
3:      $q \leftarrow$  Particione( $A, p, r$ );
4:     Quick-Sort( $A, p, q - 1$ );
5:     Quick-Sort( $A, q + 1, r$ );
6:   fim
7:
```

p					r				
99	33	55	77	11	22	88	66	33	44

Pseudo-código

```
1: Quick-Sort( $A, p, r$ )  
2:   Se  $p < r$  então  
3:      $\triangleright q \leftarrow$  Particione( $A, p, r$ );  
4:     Quick-Sort( $A, p, q - 1$ );  
5:     Quick-Sort( $A, q + 1, r$ );  
6:   fim  
7:
```

p		q				r			
33	11	22	33	44	55	88	66	77	99

Pseudo-código

```
1: Quick-Sort( $A, p, r$ )
2:   Se  $p < r$  então
3:      $q \leftarrow$  Particione( $A, p, r$ );
4:      $\triangleright$  Quick-Sort( $A, p, q - 1$ );
5:     Quick-Sort( $A, q + 1, r$ );
6:   fim
7:
```

p		q				r			
33	11	22	33	44	55	88	66	77	99

Pseudo-código

```
1: Quick-Sort( $A, p, r$ )
2:   Se  $p < r$  então
3:      $q \leftarrow$  Particione( $A, p, r$ );
4:      $\triangleright$  Quick-Sort( $A, p, q - 1$ );
5:     Quick-Sort( $A, q + 1, r$ );
6:   fim
7:
```

p		q				r			
11	22	33	33	44	55	88	66	77	99

Pseudo-código

```
1: Quick-Sort( $A, p, r$ )
2:   Se  $p < r$  então
3:      $q \leftarrow$  Particione( $A, p, r$ );
4:     Quick-Sort( $A, p, q - 1$ );
5:     ▷ Quick-Sort( $A, q + 1, r$ );
6:   fim
7:
```

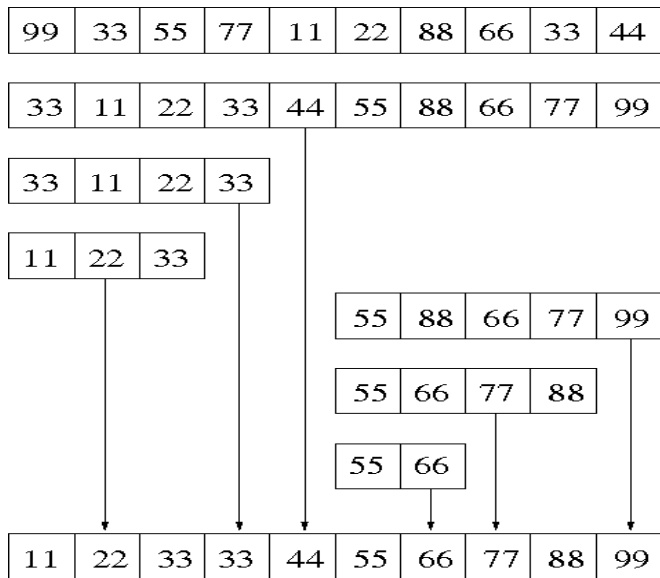
p		q				r			
11	22	33	33	44	55	88	66	77	99

Pseudo-código

```
1: Quick-Sort( $A, p, r$ )
2:   Se  $p < r$  então
3:      $q \leftarrow$  Particione( $A, p, r$ );
4:     Quick-Sort( $A, p, q - 1$ );
5:     ▷ Quick-Sort( $A, q + 1, r$ );
6:   fim
7:
```

p		q				r			
11	22	33	33	44	55	66	77	88	99

Quick Sort



Exercícios

Exercício 1) implemente o algoritmo Quick-Sort para ordenar números inteiros. Teste-o com 10, 100, 1.000, 10.000, 100.000, 1.000.000 de números aleatórios. Este algoritmo é mais rápido que os anteriores?

Exercícios

Exercício 2) Utilize o algoritmo Quick-Sort para ordenar 10, 100, 1.000, 10.000, 100.000, 1.000.000 de números:

- em ordem crescente;
- em ordem decrescente;
- iguais;

Para qual dos casos acima o algoritmo foi pior? Mostre os tempos do algoritmo, para esses 3 casos e inclusive para os números aleatórios.